

Universidad Tecnológica Nacional

Base de Datos II

Vistas

Autor: Angel Simón

Índice

1. Introducción	2
2. Sintaxis básica	2
3. Ejemplo práctico	3
4. Modificación y eliminación de vistas	5
4.1. ALTER VIEW	5
4.2. DROP VIEW	6
5. Anexo: Script de creación de la base de datos de ejemplo	6

1. Introducción

Una vista en SQL Server es un objeto de base de datos que encapsula y almacena una consulta de selección. Conceptualmente, puede entenderse como una **tabla virtual**: no almacena datos propios, sino que los obtiene dinámicamente a partir de una o varias tablas reales cada vez que es consultada.

Concepto Clave

Una vista no almacena datos. Cada vez que se la consulta, SQL Server ejecuta internamente la consulta que la define y devuelve el resultado en ese momento.

Las vistas resultan especialmente útiles en los siguientes escenarios:

- **Simplificar consultas complejas:** permiten encapsular lógica de cálculo o uniones entre tablas, exponiéndola como si fuera una tabla simple.
- **Control de acceso:** posibilitan otorgar permisos sobre la vista sin exponer las tablas subyacentes.
- **Seguridad:** permiten ocultar columnas sensibles (por ejemplo, salarios o datos personales) a ciertos usuarios.
- **Estandarización:** centralizan la lógica de consultas frecuentes, evitando la duplicación de código SQL en múltiples reportes.

Lista 1. Casos de uso habituales de las vistas

2. Sintaxis básica

La creación de una vista sigue la siguiente estructura:

```
CREATE VIEW nombreVista AS
SELECT columnas
FROM tablas
WHERE condiciones;
```

Código 1. Sintaxis general de CREATE VIEW

La definición de una vista puede incluir JOINS entre tablas, funciones de agregación y condiciones de filtrado. Una vez creada, puede consultarse mediante un SELECT ordinario, exactamente

igual que si fuese una tabla real.

3. Ejemplo práctico

Consideremos una base de datos compuesta por tres tablas: Empleados, Areas y Sueldos. Supongamos que necesitamos generar múltiples reportes que incluyan, para cada empleado, su área de pertenencia, su antigüedad en años y su último sueldo cobrado. Dado que el cálculo de estas columnas es relativamente complejo, resulta conveniente centralizarlo en una vista.

El esquema de la base de datos es el siguiente:

Tabla	Columna	Tipo
Empleados	IDEmpleado (PK)	INT NOT NULL
Empleados	Apellido	NVARCHAR(50) NOT NULL
Empleados	Nombre	NVARCHAR(50) NOT NULL
Empleados	IDArea (FK)	INT NOT NULL
Empleados	FechaIngreso	DATE NOT NULL
Areas	IDArea (PK)	INT NOT NULL
Areas	Descripcion	NVARCHAR(100) NOT NULL
Sueldos	IDEmpleado (FK)	INT NOT NULL
Sueldos	FechaPago	DATE NOT NULL
Sueldos	Sueldo	DECIMAL(10,2) NOT NULL

Tabla 1. Estructura de las tablas de la base de datos de ejemplo

Diagrama de Entidad Relación



Figura 1. Diagrama de Entidad Relación – Empleados, Areas y Sueldos

La vista que encapsula la lógica de consulta sería la siguiente. Véase el **Código 2** para el detalle completo:

```
CREATE VIEW VW_DatosEmpleado AS
SELECT
    e.IDEmpleado,
    e.Apellido,
    e.Nombre,
    a.Descripcion AS NombreArea,
    DATEDIFF(YEAR, e.FechaIngreso, GETDATE())
    - CASE
        WHEN DATEADD(YEAR, DATEDIFF(YEAR, e.FechaIngreso, GETDATE()),
            e.FechaIngreso) > GETDATE()
        THEN 1
        ELSE 0
    END AS AntiguedadAnios,
    (
        SELECT TOP 1 s.Sueldo
        FROM Sueldos s
        WHERE s.IDEmpleado = e.IDEmpleado
        ORDER BY s.FechaPago DESC
    ) AS UltimoSueldo
FROM
    Empleados e
    INNER JOIN Areas a ON e.IDArea = a.IDArea;
```

Código 2. Definición de la vista VW_DatosEmpleado

Una vez creada la vista, puede consultarse de forma directa:

```
SELECT * FROM VW_DatosEmpleado;
```

Código 3. Consulta sobre la vista

IDEmpleado	Apellido	Nombre	NombreArea	AntiguedadAnios	UltimoSueldo
101	González	María	Recursos Humanos	3	152000.00
102	Pérez	Juan	Sistemas	3	205000.00
103	López	Ana	Sistemas	2	182500.00
104	Fernández	Carlos	Contabilidad	4	172000.00
105	Ramírez	Lucía	Recursos Humanos	2	157000.00

Tabla 2. Resultado de SELECT * FROM VW_DatosEmpleado

Una de las ventajas más destacadas es que las columnas calculadas de la vista —como `UltimoSueldo`— pueden utilizarse directamente en condiciones `WHERE`, tal como si fueran columnas reales de una

tabla:

```
SELECT Apellido, Nombre, UltimoSueldo
FROM VW_DatosEmpleado
WHERE UltimoSueldo > 180000;
```

Código 4. Filtrado sobre una columna calculada de la vista

Apellido	Nombre	UltimoSueldo
Pérez	Juan	205000.00
López	Ana	182500.00

Tabla 3. Empleados cuyo ultimo sueldo sea mayor a 180000

Abstracción de la complejidad

Si en algún momento cambia la lógica de cálculo del sueldo o la antigüedad, bastará con modificar la definición de la vista. El cambio se propagará automáticamente a todos los reportes y consultas que la utilicen, sin necesidad de modificarlos individualmente.

4. Modificación y eliminación de vistas

4.1. ALTER VIEW

Para modificar la definición de una vista existente se utiliza ALTER VIEW. La sintaxis es análoga a la de CREATE VIEW:

```
ALTER VIEW VW_DatosEmpleado AS
SELECT
    e.IDEmpleado,
    e.Apellido,
    e.Nombre,
    a.Descripcion AS NombreArea
FROM
    Empleados e
    INNER JOIN Areas a ON e.IDArea = a.IDArea;
```

Código 5. Modificación de una vista con ALTER VIEW

Tras ejecutar este `ALTER VIEW`, la vista quedará redefinida: al consultarla, solo devolverá las columnas `IDEmpleado`, `Apellido`, `Nombre` y `NombreArea`, sin los campos calculados anteriores.

4.2. DROP VIEW

Para eliminar definitivamente una vista de la base de datos se utiliza `DROP VIEW`:

```
DROP VIEW VW_DatosEmpleado;
```

Código 6. Eliminación de una vista con DROP VIEW

Una vez ejecutado, el objeto deja de existir en la base de datos y cualquier consulta que intente acceder a él fallará con un error.

Dependencias entre objetos

Si otras vistas, procedimientos almacenados o funciones hacen referencia a la vista que se está eliminando, éstos también dejarán de funcionar correctamente. Antes de ejecutar un `DROP VIEW`, verifíca que no existan dependencias activas sobre el objeto.

5. Anexo: Script de creación de la base de datos de ejemplo

A continuación se incluye el script SQL completo para recrear la base de datos utilizada en los ejemplos anteriores, incluyendo la estructura de tablas y los datos de prueba.

```
CREATE DATABASE EjemploVista
COLLATE Latin1_General_CI_AI
GO

USE EjemploVista
GO

CREATE TABLE Areas (
    IDArea INT PRIMARY KEY,
    Descripcion NVARCHAR(100) NOT NULL
);
```

```

CREATE TABLE Empleados (
    IDEmpleado INT PRIMARY KEY,
    Apellido NVARCHAR(50) NOT NULL,
    Nombre NVARCHAR(50) NOT NULL,
    IDArea INT NOT NULL,
    FechaIngreso DATE NOT NULL,
    FOREIGN KEY (IDArea) REFERENCES Areas(IDArea)
);

CREATE TABLE Sueldos (
    IDEmpleado INT NOT NULL,
    FechaPago DATE NOT NULL,
    Sueldo DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (IDEmpleado, FechaPago),
    FOREIGN KEY (IDEmpleado) REFERENCES Empleados(IDEmpleado)
);

```

Código 7. Creación de la base de datos EjemploVista

```

INSERT INTO Areas VALUES (1, 'Recursos Humanos');
INSERT INTO Areas VALUES (2, 'Sistemas');
INSERT INTO Areas VALUES (3, 'Contabilidad');

INSERT INTO Empleados VALUES (101, 'González', 'María', 1, '2022-03-15');
INSERT INTO Empleados VALUES (102, 'Pérez', 'Juan', 2, '2021-07-01');
INSERT INTO Empleados VALUES (103, 'López', 'Ana', 2, '2023-01-10');
INSERT INTO Empleados VALUES (104, 'Fernández', 'Carlos', 3, '2020-11-20');
INSERT INTO Empleados VALUES (105, 'Ramírez', 'Lucía', 1, '2022-05-05');

INSERT INTO Sueldos VALUES (101, '2024-03-31', 150000.00);
INSERT INTO Sueldos VALUES (101, '2024-04-30', 152000.00);
INSERT INTO Sueldos VALUES (102, '2024-03-31', 200000.00);
INSERT INTO Sueldos VALUES (102, '2024-04-30', 205000.00);
INSERT INTO Sueldos VALUES (103, '2024-03-31', 180000.00);
INSERT INTO Sueldos VALUES (103, '2024-04-30', 182500.00);
INSERT INTO Sueldos VALUES (104, '2024-03-31', 170000.00);
INSERT INTO Sueldos VALUES (104, '2024-04-30', 172000.00);
INSERT INTO Sueldos VALUES (105, '2024-03-31', 155000.00);
INSERT INTO Sueldos VALUES (105, '2024-04-30', 157000.00);

```

Código 8. Inserción de datos de prueba